

Overview

This project automates the process of collecting spectral data across a 2D surface using two motorized Thorlabs ELL17 linear stages and an Ocean Optics HR400 spectrometer.

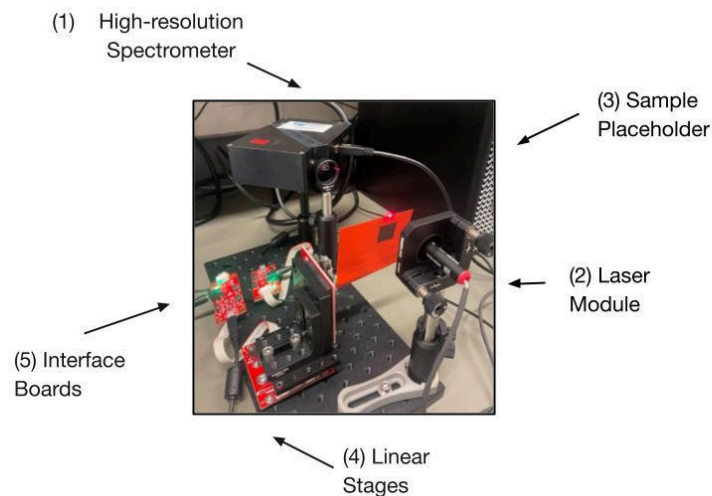
The system can:

- Move to a single target point and record a spectrum.
- Perform a snake-pattern scan over a defined grid, capturing spectra at each position.

Spectra are automatically saved in .csv format for later processing, and a separate plotting script can generate heatmaps for a chosen wavelength.

Hardware Setup

The experimental setup consisted of five primary components:



Photograph of the complete optical scanning system with labeled components.

Component	Model	Key Specs	Function
Spectrometer	Ocean Optics HR400	3,401 channels, ~200-1,100 nm, 0.1 nm resolution (nominal)	Capturing of full spectra at each position.
Laser Module	Thorlabs, 650 nm, 1 mW	Class II, continuous-wave	Illumination for reflective measurements.
Linear Stages (2x)	Thorlabs ELL17	28 mm travel, piezoelectric drive, ~100 μ m repeatability	Positioning of sample in both X & Y directions.
Interface Boards	Thorlabs ELL17	USB 2.0	Communication with

	Interface		stages.
Sample	Adhesive paper note	—	Simulating surfaces with varying transmission properties.

Software Guide

The software is split into **four main scripts**:

1. **linear_stage_x.py** / **linear_stage_y.py**: Control classes for the two stages.
2. **main.py**: Stage connections, spectrometer setup, scanning logic, and saving spectra.
3. **gui.py**: Graphical interface for controlling scans without editing code.
4. **plot_spectra.py**: Loads saved spectra and generates heatmaps.

Install the following Python packages:

```
pip install elliptec seabreeze pyqt5 pandas numpy matplotlib
```

Linear Stage (X)

```
# linear_stage_x.py
# Handles communication with the X-axis Thorlabs ELL17 linear stage

import elliptec
import sys, os

class LinearStageX():
    # Automatically initializes variables when the object is created
    def __init__(self):
        self.ports = None          # List of available COM ports
        self.device = None         # The connected stage device object
        self._controller = None    # Controller object for sending commands

    def find_devices(self):
        # Scan the computer for connected ELL17 controllers
        ports = elliptec.scan.find_ports()
        self.ports = ports[:]      # Store ports in object variable
        return ports
```

```

def connect(self, idn):
    # Create a controller for the given COM port
    self._controller = elliptec.Controller(idn, debug=True)
    self._controller.port = idn

    # Find devices connected to the controller
    self.devices = elliptec.scan_for_devices(controller=self._controller)

    # Create a Linear stage object for control
    self.device = elliptec.Linear(self._controller)
    return self.device.info # Return device information

def move(self, pos):
    # Move the stage to a specific position (mm)
    sys.stdout = open(os.devnull, 'w') # Suppress stage output
    position = self.device.set_distance(pos)
    sys.stdout = sys.__stdout__
    return position

```

Linear Stage (Y)

```

# linear_stage_y.py
# Handles communication with the Y-axis Thorlabs ELL17 linear stage
# Almost identical to LinearStageX, but separated for clarity

import elliptec
import sys, os

class LinearStageY():
    def __init__(self):
        self.ports = None
        self.device = None
        self._controller = None

    def find_devices(self):
        ports = elliptec.scan.find_ports()
        self.ports = ports[:]
        return ports

```

```

def connect(self, idn):
    self._controller = elliptec.Controller(idn, debug=True)
    self._controller.port = idn
    self.devices = elliptec.scan_for_devices(controller=self._controller)
    self.device = elliptec.Linear(self._controller)
    return self.device.info

def move(self, pos):
    sys.stdout = open(os.devnull, 'w')
    position = self.device.set_distance(pos)
    sys.stdout = sys.__stdout__
    return position

```

Main Script

This script:

1. Connects to the two linear stages.
2. Connects to the spectrometer.
3. Performs a snake scan (alternating X direction each Y step).
4. Saves each spectrum as a CSV file.

Import and Initialize

```

# main.py
# Coordinates scanning between the X/Y linear stages and spectrometer

import seabreeze.spectrometers as sb # Ocean Optics spectrometers
import time
import os
from PyQt5 import QtWidgets

# Import our stage classes
from linear_stage_x import LinearStageX
from linear_stage_y import LinearStageY

# Initialize stage objects
x_stage = LinearStageX()
y_stage = LinearStageY()

```

```

# Find and connect to stages
ports_x = x_stage.find_devices()
ports_y = y_stage.find_devices()

x_stage.connect('COM5') # Replace with your X-axis COM port
y_stage.connect('COM6') # Replace with your Y-axis COM port

# Initialize spectrometer
devices = sb.list_devices()
spectrometer = sb.Spectrometer.from_first_available()
spectrometer.integration_time_micros(100_000) # Exposure time in microseconds

```

Creating a Folder for Each Scan Run

```

# Creates a new folder for each scan, incrementing run numbers
def get_new_run_directory(base="spectra"):
    os.makedirs(base, exist_ok=True)
    existing_runs = [d for d in os.listdir(base) if d.startswith("run_") and
os.path.isdir(os.path.join(base, d))]
    run_numbers = [int(name.split("_")[1]) for name in existing_runs if
name.split("_")[1].isdigit()]
    next_run_number = max(run_numbers, default=0) + 1
    run_dir = os.path.join(base, f"run_{next_run_number}")
    os.makedirs(run_dir)
    return run_dir

```

Stop Button Component

```

stop_requested = False

def request_stop():
    global stop_requested
    stop_requested = True

```

Scan To (Snake Scan)

```

# Moves the Y stage in steps, scanning X in opposite directions each row
def scan(x_stage, y_stage, spectrometer,
        x_start=0, x_end=28, x_step=2,
        y_start=0, y_end=28, y_step=2,
        wait_time=0.1, gui=None):

    global stop_requested
    stop_requested = False

    run_dir = get_new_run_directory()

    y_positions = list(range(y_start, y_end + 1, y_step))
    for i, y_pos in enumerate(y_positions):
        if stop_requested:
            print("Scan stopped by user.")
            return

        y_stage.move(y_pos)
        print(f"Moved Y to {y_pos}")
        time.sleep(wait_time)

        if gui is not None:
            gui.y_pos.setText(str(y_pos))
            QtWidgets.QApplication.processEvents()

        # Alternate X direction each row
        x_positions = list(range(x_start, x_end + 1, x_step)) if i % 2 == 0 else
list(range(x_end, x_start - 1, -x_step))

        for x_pos in x_positions:
            if stop_requested:
                print("Scan stopped by user.")
                return

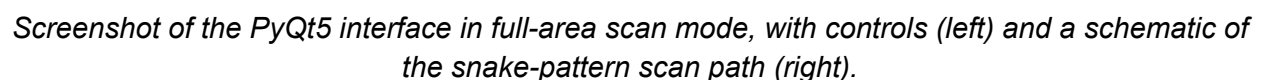
            x_stage.move(x_pos)
            print(f"Moved X to {x_pos}")
            time.sleep(wait_time)

            if gui is not None:
                gui.x_pos.setText(str(x_pos))

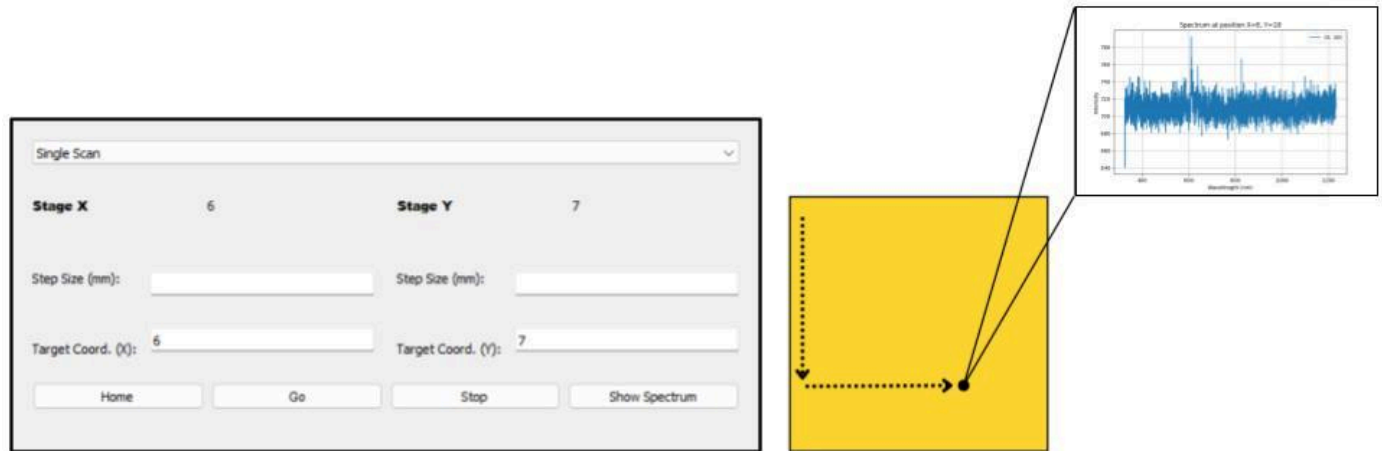
```

Design PyQt GUI

The GUI incorporates two primary scanning modes. “**Scan To**” mode executes an automated sweep of the defined area in a continuous snake-like pattern. Users specify the “target” or endpoint coordinates and the step size, which determines the format of the scan.



“Single Scan” mode captures a spectrum at a specific location. Users input the target coordinates, and the stages position the spectrometer accordingly.



Screenshot of the PyQt5 interface in single-point scan mode, with coordinate entry (left) and schematic showing direct point targeting (right).

For both modes, additional control buttons include:

- **Home:** returns the stages to the origin at coordinates (0,0).
- **Stop:** immediately halts movement.
- **Go:** initiates the scan.
- **Show Spectrum:** plots the wavelength versus intensity graph for the current position.

Connect GUI and Code

```
from PyQt5.QtWidgets import QWidget
from PyQt5.uic import loadUi

# Import core objects and functions from main.py
from main import stage_x, stage_y, spec, scan, move_to_position, request_stop

# Custom QWidget subclass to control the linear stages via GUI
class StageWidget(QWidget):

    # Initialize the widget and load the UI design from a .ui file
    def __init__(self):
        super().__init__()

        loadUi('./test.ui', self) # Loads the UI layout created in Qt Designer

    # Connect button click events to their handler methods
```



```

self.home.clicked.connect(self.home_clicked)
self.go.clicked.connect(self.go_clicked)
self.stop.clicked.connect(self.stop_clicked)

# Handler for the "Home" button: moves both stages to position (0, 0)
def home_clicked(self):
    move_to_position(stage_x, stage_y, 0, 0)
    # Update displayed position labels
    self.x_pos.setText("0")
    self.y_pos.setText("0")

# Handler for the "Go" button: either moves to a single point or starts a scan
def go_clicked(self):
    # Read target positions from input fields as integers
    x_target = int(self.x_target.text())
    y_target = int(self.y_target.text())

    # Get the current selected mode from dropdown (e.g., "Single Scan" or
    "Snake Scan")
    mode = self.mode_selection.currentText()

    if mode == "Single Scan":
        # Move stages once to the specified (x_target, y_target) coordinates
        move_to_position(stage_x, stage_y, x_target, y_target)
        # Update position labels accordingly
        self.x_pos.setText(str(x_target))
        self.y_pos.setText(str(y_target))
    else:
        # For scanning mode, get step sizes for X and Y directions
        x_step = int(self.x_step.text())
        y_step = int(self.y_step.text())

        # Start the scanning routine with specified parameters
        scan(stage_x, stage_y, spec,
            x_start=0, x_end=x_target, x_step=x_step,
            y_start=0, y_end=y_target, y_step=y_step,
            gui=self) # Pass self to allow GUI position updates

# Handler for the "Stop" button: signals the scan to stop gracefully
def stop_clicked(self):
    print("Stop button clicked")
    request_stop()

```

```

def show_clicked(self):
    x_target = int(self.x_target.text())
    y_target = int(self.y_target.text())

    # Find the most recent run folder
    run_dirs = sorted(glob.glob("spectra/run_*"), key=os.path.getmtime)
    if not run_dirs:
        print("No run folders found.")
        return
    latest_run = run_dirs[-1]

    # Build expected file name
    file_path = os.path.join(latest_run, f"spectrum_y{y_target}_x{x_target}.csv")

    if not os.path.exists(file_path):
        print(f"No spectrum found for position ({x_target}, {y_target}).")
        return

    # Load and plot spectrum
    df = pd.read_csv(file_path)
    plt.figure(figsize=(8, 5))
    plt.plot(df["Wavelength"], df["Intensity"], label=f"({x_target}, {y_target})")
    plt.xlabel("Wavelength (nm)")
    plt.ylabel("Intensity")
    plt.title(f"Spectrum at position X={x_target}, Y={y_target}")
    plt.legend()
    plt.grid(True)
    plt.show()

from PyQt5.QtWidgets import QMainWindow

# Main application window class that holds the StageWidget
class StageGui(QMainWindow):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("Stage Controller")

        # Instantiate the StageWidget and set it as the central widget
        self.stage_widget = StageWidget()
        self.setCentralWidget(self.stage_widget)

```

```

        self.show()

    # Properly handle closing the widget and then the main window
    def closeEvent(self, *args, **kwargs):
        self.stage_widget.close()
        super(QMainWindow, self).closeEvent(*args, **kwargs)

import sys
from PyQt5.QtWidgets import QApplication

# Standard Qt application entry point
if __name__ == "__main__":
    app = QApplication(sys.argv)
    window = StageGui()
    sys.exit(app.exec_())

```

Plotting Spectral Data

Once the scan is finished, we will have a folder of CSV files with names like:

spectrum_y10_x6.csv

Each contains two columns:

- **Wavelength** (nm)
- **Intensity** (counts)

We can load these files into Python and plot them to visualize either:

- A single point's spectrum (Show Spectrum function).
- The change in intensity across the scan (Heatmap).

```

# plot_spectra.py
# Reads all saved spectrum CSV files from a given run, extracts the intensity
# at a target wavelength, and visualizes the values as a heatmap using matplotlib.

import os
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

```

```

# Set base folder, run folder, and target wavelength for visualization
base_dir = "spectra"
run_name = "run_20250746"
run_path = os.path.join(base_dir, run_name)
target_wavelength = 638 # wavelength (nm) to extract from each spectrum

# Dictionary to store spectra for each (x, y) position
data = {}
for x in range(29):
    for y in range(29):
        # Build the file path for the current spectrum
        filepath = os.path.join(run_path, f"spectrum_y{y}_x{x}.csv")
        if os.path.exists(filepath):
            df = pd.read_csv(filepath) # load spectrum into DataFrame
            data[(x, y)] = df # store spectrum with its coordinates

# Matrix to store extracted intensity values for each (y, x) position
intensity_map = np.zeros((29, 29))

# Extract intensity at the closest wavelength to the target for each spectrum
for (x, y), df in data.items():
    idx = (df["Wavelength"] - target_wavelength).abs().idxmin() # find closest
wavelength index
    intensity_map[y, x] = df.loc[idx, "Intensity"] # assign intensity
to matrix

# Plot heatmap (flipped left-right for display purposes)
plt.imshow(np.fliplr(intensity_map), cmap='hot', origin='lower')

plt.title(f"Heatmap at {target_wavelength} nm")
plt.colorbar(label="Intensity")
plt.xlabel("X Position")
plt.ylabel("Y Position")
plt.show()

```

How to Run the Project

Follow these steps to run the full workflow:

1. **Connect hardware**

- Plug in the spectrometer and verify it is detected by **seabreeze**.
 - Connect both linear stages via USB and confirm their COM ports.
2. **Run the GUI**
`python gui.py`
 3. **View results**
Use `plot_spectra.py` to visualize the saved spectra.

Key Notes

- The **linear stage control logic** here is based on the "Linear Stage" section, but extended for two-axis scanning.
- The **snake scan** method reduces total travel time by reversing the X direction every Y step instead of resetting.
- For best performance, adjust:
 - **Integration time** of the spectrometer (`spectrometer.integration_time_micros()`).
 - **Step size** of the stages (`x_step`, `y_step`).
 - **Wait time** after stage movement to allow vibration settling.
- Full code available at: <https://github.com/lyknj/spectrometer-stage-control>

About Me

Hello! My name is Anna Luo, and I'm a high school senior at Canyon Crest Academy in San Diego, California.

I had the privilege of working in the photonics lab at the ASRC under Professor Viktoriia Rutckaia as part of the NYU GSTEM summer program. Now, I hope this tutorial will help others build and use similar scanning setups for their own research and projects.

